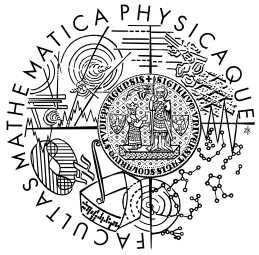

FACULTY
OF MATHEMATICS
AND PHYSICS
Charles University

BACHELOR THESIS

Kinan Al-Falakh

Automating Benchmark Analysis Through Machine Learning

Department of Distributed and Dependable Systems

Supervisor of the bachelor thesis: Mgr. Vojtěch Horký, Ph.D.

Study programme: Computer Science

Prague 2026

I declare that I carried out this bachelor thesis on my own, and only with the cited sources, literature and other professional sources. I understand that my work relates to the rights and obligations under the Act No. 121/2000 Sb., the Copyright Act, as amended, in particular the fact that the Charles University has the right to conclude a license agreement on the use of this work as a school work pursuant to Section 60 subsection 1 of the Copyright Act.

In ..... date ....

Author's signature

Dedication.

Title: Automating Benchmark Analysis Through Machine Learning

Author: Kinan Al-Falakh

Department: Department of Distributed and Dependable Systems

Supervisor: Mgr. Vojtěch Horký, Ph.D., Department of Distributed and Dependable Systems

Abstract: Regular software performance benchmarking is used to prevent silent performance deterioration, yet reliably evaluating the results remains an open challenge. This thesis employs contrastive time series representation learning via TS2Vec to automate the detection of performance anomalies in GraalVM compiler benchmarks. Raw iteration times are normalized using a log-transform followed by robust IQR scaling to handle multiplicative noise from garbage collection and compiler de-optimizations. A single dilated convolutional encoder, trained configuration-independently with hierarchical temporal contrastive loss, maps variable-length benchmark runs to fixed-size embedding vectors. Performance anomalies are then detected as unusual transitions in embedding space using cosine distance between consecutive versions. The method is evaluated on GraalVM benchmark data across multiple domain-specific configurations.

Keywords: machine learning, software benchmarking, automated analysis, contrastive learning, time series

Contents

Introduction

Systematic testing of software performance during development is a persistent challenge. Performance anomalies, changes that silently alter performance characteristics, can go unnoticed if benchmark results are not evaluated regularly. In large-scale software such as the GraalVM compiler, where benchmark data spans years of development and tens of thousands of measurement runs, reliably automated evaluation of the results becomes essential. While collecting sufficient measurement data is itself costly, another hard problem is interpreting it: software rarely behaves in a fully deterministic manner, and non-deterministic behavior in the runtime environment introduces noise into iteration times, making it difficult to determine whether a change in measured performance reflects a genuine anomaly or is simply a consequence of the execution environment.

Existing approaches to automated performance evaluation range from statistical change detection to classical unsupervised learning. These methods typically require significant manual configuration per benchmark or assume stationary data distributions, limiting their applicability to large-scale, long-running projects.

This thesis explores whether machine learning, specifically self-supervised contrastive representation learning, can better distinguish genuine performance anomalies from noise. We build and evaluate the system on the GraalVM Compiler Benchmark Results Dataset, a data artifact collected by the D3S department at Charles University together with Oracle Labs to support research into software performance testing. We train a neural encoder to learn what benchmark execution "looks like" from unlabeled data, and detect anomalies as unusual shifts in the learned representation space. The goal is to provide performance engineers with a compact set of flagged transitions that merit further investigation, reducing the volume of results requiring manual inspection.

The remainder of this thesis is organized as follows. Chapter 1 provides background on software performance benchmarking and JVM execution characteristics, introduces the GraalVM benchmark dataset, defines the notion of a configuration, and formulates the problem addressed by this thesis. Chapter 2 describes the preprocessing pipeline, including steady-state detection and the normalization strategy used to make the data robust to runtime noise and comparable across benchmarks. Chapter 3 surveys related work in performance regression detection and time series representation learning. Chapter 4 presents the TS2Vec contrastive learning framework and how it is applied to benchmark time series. Chapter 5 describes the anomaly detection pipeline, the multi-layer filtering system, and the reporting output designed for performance engineers.

1 Background, Dataset, and Problem Statement

This chapter provides the necessary background for the remainder of the thesis. We begin with an overview of software performance benchmarking and its role in software development. We then describe the execution characteristics of the Java Virtual Machine that make benchmark interpretation challenging. Finally, we introduce the GraalVM Compiler Benchmark Results Dataset, define the notion of a *configuration* as the fundamental unit of analysis, and formulate the problem addressed by this thesis.

1.1 Software Performance Benchmarking

Software performance benchmarking is the practice of executing standardized workloads under controlled conditions to measure and compare the performance of a software system across different versions, configurations, or environments. In the context of software development, benchmarking serves a critical role: it provides an objective basis for detecting performance regressions before they reach production.

A typical benchmarking workflow consists of three stages. First, a set of representative workloads is selected and executed on the software under test. Second, performance metrics such as execution time, throughput, or resource utilization are recorded for each execution. Third, the recorded metrics are compared against a baseline, typically the measurements from a previous software version, to determine whether a statistically significant change has occurred.

Each of these stages presents its own challenges. Designing efficient measurement procedures, selecting representative workloads, and managing the cost of collecting data across many software versions all require significant effort (1). However, the evaluation and interpretation of the results poses a particular difficulty because software rarely behaves in a fully deterministic manner. Even when the same code runs on the same hardware, consecutive executions can produce measurably different results due to variability in the runtime environment, hardware state, and operating system behavior. Distinguishing a genuine performance change from such variability is one of the key challenges addressed in this thesis.

The cost of benchmarking at scale is also substantial. In large projects where benchmarks must be executed across many software versions and hardware configurations, the total measurement time can reach years of machine time. This makes it impractical to simply increase the number of repetitions until statistical significance is achieved for every transition, and further emphasizes the need for methods that can reliably evaluate benchmark results automatically.

1.2 JVM Execution Characteristics

The benchmarks used in this work execute on the Java Virtual Machine (JVM). The JVM introduces several sources of non-deterministic behavior that

directly affect benchmark measurements. Understanding these sources is essential for interpreting the data and motivating the preprocessing steps described in Chapter 2.

1.2.1 JIT Compilation

The JVM supports multiple compilation strategies. In its default mode, it uses just-in-time (JIT) compilation, where bytecode is initially interpreted and only compiled to native machine code after it has been identified as *hot*, one that is executed frequently enough to justify the cost of compilation. More recent versions of the GraalVM Compiler also support ahead-of-time (AOT) compilation of Java applications into native binaries (1).

Modern JVMs such as those using the GraalVM compiler employ tiered compilation (2), where methods progress through multiple compilation levels of increasing optimization aggressiveness. The JVM profiles methods during interpretation and uses the collected runtime information to guide the compiler in producing optimized native code.

Crucially, because the compiler and the application share the same execution resources (processor and memory), the timing of compilation directly affects the performance observed during benchmark execution. Early iterations of a benchmark workload execute partially interpreted or minimally optimized code, resulting in higher execution times. As compilation progresses, execution times decrease until the application reaches a state commonly referred to as *steady state*, where the most frequently executed methods have been compiled and optimized. However, even during steady state, the compiler may occasionally *de-optimize* previously compiled methods when runtime assumptions are invalidated, triggering recompilation and causing temporary performance disruptions.

Furthermore, as noted by Bulej et al. (1), profile-guided compilation is non-deterministic in principle. A single execution is not necessarily representative of the average performance observable with the same benchmark workload and compiler version across multiple executions.

1.2.2 Garbage Collection

The JVM provides automatic memory management through a mechanism known as garbage collection (GC). This process periodically reclaims memory that is no longer in use by the application. Garbage collection can introduce pauses at any point during execution, including during the steady-state phase. These pauses are a source of non-determinism in benchmark measurements and are accounted for as part of the normalization strategy described in Chapter 2.

The dataset includes measurements collected under multiple GC configurations, allowing the analysis pipeline to identify whether a detected anomaly is specific to a particular configuration or is observed across multiple configurations.

1.2.3 Typical Benchmark Execution Profile

A benchmark *run* is a single execution of a benchmark workload. During a run, the workload is repeated multiple times, and each repetition is called an

iteration. The wall-clock time of each iteration, referred to as the *iteration time*, is the primary metric of interest throughout this thesis.

A typical benchmark run on the JVM can be divided into two phases (3). The *warmup phase* is characterized by high and variable iteration times as the JVM actively compiles hot methods. The *steady-state phase* begins once major compilation activity has subsided and iteration times stabilize, though they remain subject to occasional spikes. Traini et al. (3) note that some benchmarks may take a very long time to stabilize or may not stabilize at all.

Figure 1.1 illustrates both phases across four benchmark runs from the dataset: the initial high iteration times during warmup followed by a more stable regime with periodic spikes. Note that the second run in the figure, despite appearing visually noisy, is still considered to be in steady state — the spiky behavior reflects the nature of the corresponding benchmark.

For performance evaluation, only the steady-state iterations are relevant. The warmup phase reflects JVM runtime behavior rather than compiled code quality. Identifying the transition point is the task of the steady-state detector described in Chapter 2.

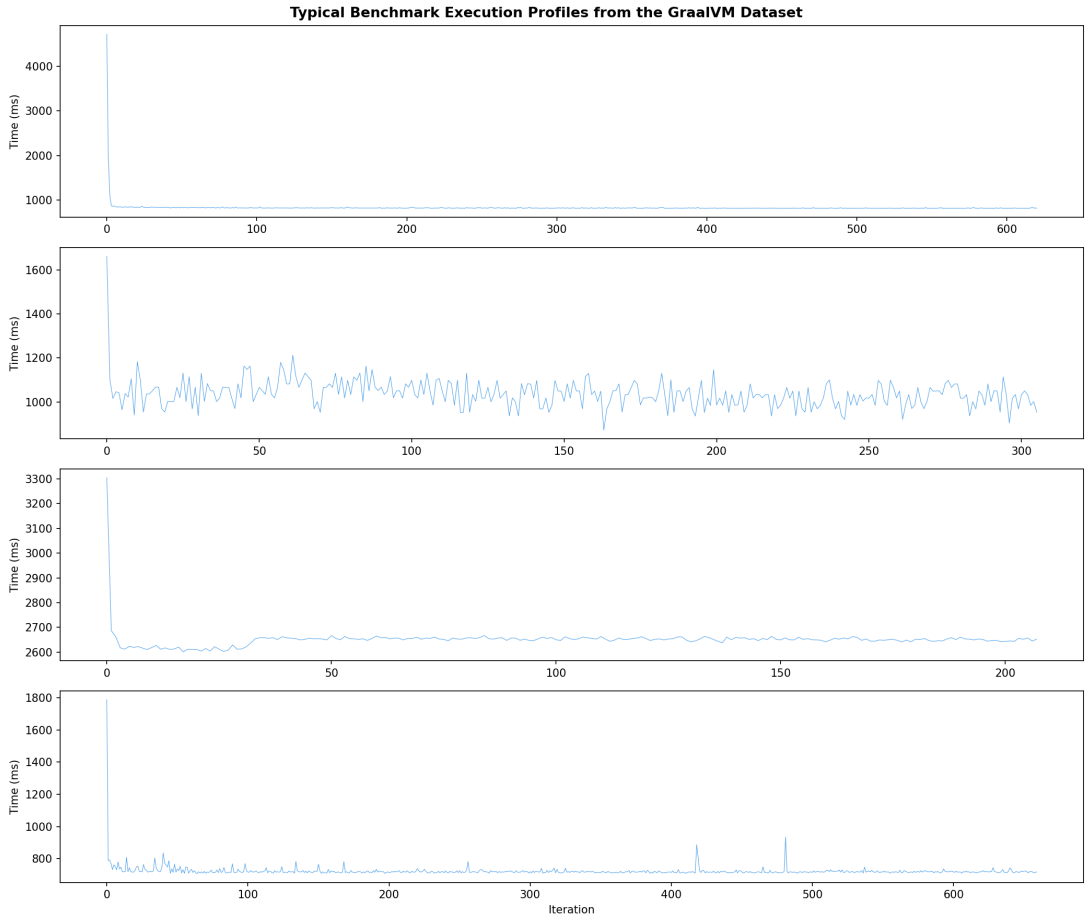

Figure 1.1 Raw iteration times for four benchmark runs from the GraalVM dataset. The warmup phase (high initial iteration times) and the subsequent steady-state phase with periodic noise spikes are visible across the benchmarks.

1.3 The GraalVM Compiler Benchmark Results Dataset

The data used throughout this thesis comes from the GraalVM Compiler Benchmark Results Dataset, a publicly available data artifact presented by Bulej et al. (1). The dataset aggregates more than 70 machine time years of performance measurements collected over seven years of development of the GraalVM Compiler Project, from 2016 to 2022. It was created by researchers at the D3S department of Charles University together with engineers at Oracle Labs, with the explicit goal of supporting research into performance change detection methods and related problems. The benchmark workloads come from well-known suites including DaCapo (4), ScalaBench (5), a modified version of SPECjvm2008, and the Renaissance suite (6), chosen to provide representative compiler workloads. A detailed description of the measurement procedure and experiment design can be found in the same work.

1.3.1 Dataset Structure and Metadata

Each measurement in the dataset records a run of a benchmark along with associated data. The dataset also records compilation activity per iteration, which is used by the steady-state detector described in Chapter 2.

Each measurement is annotated with metadata that identifies unique properties of the benchmark run. This metadata allows us to distinguish which measurements are comparable and which are not, a distinction that is important to the analysis approach described in the next section. The full metadata schema is described in the dataset paper. Chapter 5 will provide the relevant details and explain how the pipeline resolves and processes it.

1.4 Configuration as the Unit of Analysis

As noted above, not all measurements in the dataset are directly comparable: a change in iteration time between two compiler versions could reflect an actual performance change, or it could simply be an artifact of running on a different machine or under a different garbage collection setting. To isolate the effect of compiler version changes, we need a structured way to group only those measurements that share the same experimental conditions. This is the role of the *configuration*.

1.4.1 Defining a Configuration

Throughout this thesis, a *configuration* is defined as a unique combination of four attributes:

1. **Benchmark type**: the name of the benchmark (e.g., `batik-small`).
2. **Machine host**: the identifier of the physical machine.
3. **Platform type**: the compiler build variant, combining the edition (Enterprise or Community), branch (master or release), and JDK version.

4. **GC configuration**: the garbage collection configuration.

Together, these four attributes represent the most granular level at which performance measurements can be meaningfully compared. We formalize the configuration as follows:

Definition. *Let m be a measurement. The configuration of m is the tuple $c(m) = (b, h, p, g)$ where b is the benchmark type, h is the machine host, p is the platform type, and g is the GC configuration.*

Property. *Two measurements m_1 and m_2 belong to the same configuration if and only if $c(m_1) = c(m_2)$.*

1.4.2 Metadata Resolution and Filtering

To construct a configuration, the four attributes are extracted from each measurement's metadata. Since the dataset contains measurements for both JIT and AOT (Native Image) compilation modes, and this thesis focuses on JIT compilation (Section 1.2.1), the resolution process filters out all AOT-related platform types and GC configurations. The implementation details of this process are described in Chapter 5.

1.4.3 From Configuration to Time Series

As discussed in Section 1.2.1, profile-guided compilation is non-deterministic, and the dataset may therefore contain multiple measurements for the same configuration and compiler version. After applying deduplication (retaining only the measurement with the most recorded iterations), we obtain the following:

Corollary. *Two measurements belong to the same configuration if and only if they share all four attributes and differ only in the compiler version.*

Given measurements $m_1, m_2, \ldots, m_n$ with corresponding compiler versions $v_1 < v_2 < \cdots < v_n$ such that $c(m_i) = c(m_j)$ for all i, j, they belong to the same configuration $C = c(m_i)$ and differ only in their compiler version. We define the *configuration time series* as:

$$S_C = \Big[(m_1, v_1),\ (m_2, v_2),\ \ldots,\ (m_n, v_n) \Big]$$

Each element (m_i, v_i) represents a benchmark measurement under configuration C at compiler version v_i. This sequence S_C is the fundamental input to the analysis pipeline described in this thesis.

1.5 Summary and Problem Formulation

This chapter introduced the background necessary for the remainder of this thesis. We described the sources of non-deterministic behavior in JVM benchmark execution, including JIT compilation (Section 1.2.1) and garbage collection (Section 1.2.2), and presented the GraalVM Compiler Benchmark Results Dataset (Section 1.3). We then defined the *configuration* as the unit of analysis and showed

how, after metadata resolution, filtering, and deduplication, each configuration yields an ordered sequence of measurements:

$$S_C = \Big[(m_1, v_1),\ (m_2, v_2),\ \ldots,\ (m_n, v_n) \Big]$$

Since the configuration is held constant across S_C, any performance change detected between consecutive elements (m_i, v_i) and (m_{i+1}, v_{i+1}) can be attributed to the version transition $v_i \to v_{i+1}$. The goal of this thesis is to automatically identify such transitions where a meaningful change in benchmark behavior has occurred.

However, the raw iteration times in S_C are not yet suitable for analysis: they contain warmup artifacts and are subject to noise. Chapter 2 addresses how these raw measurements are prepared for the anomaly detection pipeline.

2 Preprocessing

As established in Chapter 1, the analysis pipeline operates on the configuration time series S_C, where each element (m_i, v_i) is a benchmark measurement at a specific compiler version. However, the raw iteration times recorded in each measurement are not directly suitable for analysis. They contain warmup artifacts from JIT compilation and are subject to noise from garbage collection, de-optimizations, and other runtime factors.

This chapter describes the two preprocessing steps applied to the raw data before it enters the anomaly detection pipeline: steady-state detection, which removes the warmup phase from each measurement, and normalization, which transforms the remaining iteration times into a representation that is robust to runtime variability and comparable across measurements.

2.1 Steady-State Detection

As described in Section 1.2.3, a benchmark execution on the JVM begins with a warmup phase during which the compiler is actively optimizing hot methods. The warmup iterations are not representative of the peak performance we are interested in analyzing, and including them would obscure the actual performance characteristics of the compiled code. The goal of steady-state detection (SSD) is to identify the iteration index at which warmup has ended and discard all preceding iterations.

The approach used in this work is based on monitoring the compilation activity signal, as suggested by Bulej et al. (1), who note that "waiting for a window of reduced compilation activity, as indicated by the reported compiler thread execution duration, provided reasonably reliable results." As mentioned in Section 1.3.1, the dataset records compilation activity per iteration. By detecting when this activity falls below a threshold, we identify the approximate start of the steady-state phase. As noted in Section 1.2.3, some benchmarks may never fully reach a stable steady state, so this detection is an approximation rather than a guarantee.

2.1.1 CAS Analysis (Compilation Activity Signal)

The Compilation Activity Signal (CAS) analysis operates on the compilation time series described above. The idea is straightforward: during warmup, the compiler is actively working and the compilation time per iteration is high. Once the most important methods have been compiled, this activity drops significantly compared to the warmup phase. The CAS analysis detects this transition.

The algorithm computes a rolling mean of the compilation time over a sliding window of size w and finds the first position where it falls at or below a threshold τ. This position is the cutoff index:

$$i^* = \min\left\{ k : \frac{1}{w} \sum_{j=k}^{k+w-1} x_j \ \leq \ \tau \right\}$$

Both w and τ are computed dynamically per measurement, as described in Section 2.1.2. All iterations before i^* are discarded. If no such index exists (compilation activity never subsides), the measurement is excluded from analysis.

Figure 2.1 illustrates the CAS analysis on a benchmark run from the dataset. The compilation time is high during the first iterations and then drops, with the rolling mean crossing the threshold at the detected cutoff point.

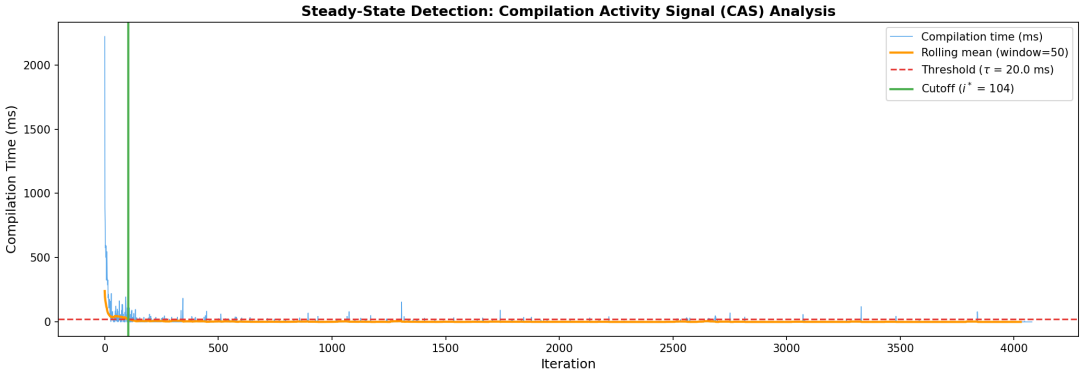

Figure 2.1 Steady-state detection on a benchmark run. The blue line shows the raw compilation time per iteration, the orange line shows the rolling mean, and the dashed red line indicates the threshold. The vertical green line marks the detected cutoff index i^*.

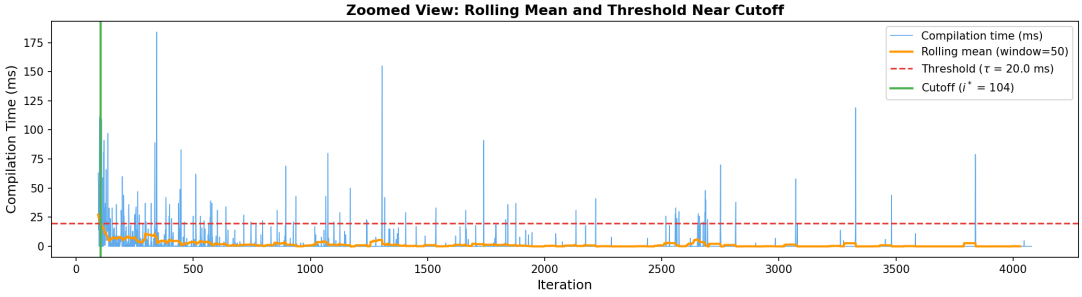

Figure 2.2 Zoomed view of the cutoff region from Figure 2.1, showing the rolling mean crossing the threshold τ at the detected cutoff point i^*.

2.1.2 Dynamic Parameters

The CAS analysis requires two parameters: the window size w for the rolling mean, and the threshold τ against which the rolling mean is compared. Choosing appropriate values for these parameters is a challenging task, as the measurements in the dataset vary widely in length (from tens to hundreds of iterations) and in the magnitude and duration of their warmup phases. Fixed parameter values would inevitably be too aggressive for some benchmarks and too conservative for others.

To address this, both parameters are computed dynamically based on properties of each individual measurement. The specific formulas were determined through *expert judgement*, by manually evaluating different parameter choices across a

variety of benchmarks with different execution profiles and selecting the values that yielded the most consistent results.

Window size w. The window size determines how many consecutive iterations are averaged when computing the rolling mean. It is set to 10% of the sequence length n, clamped to the range $[5, 50]$:

$$w = \text{clip}(0.10 \times n,\ 5,\ 50)$$

Averaging over a window rather than checking individual iterations prevents the algorithm from triggering on a single low-compilation iteration that happens to be surrounded by high ones.

Threshold τ. The threshold determines how low the rolling mean of compilation activity must drop before steady state is declared. Since different benchmarks exhibit different levels of compilation intensity, the threshold is defined relative to the maximum compilation time observed in the measurement:

$$\tau = 0.009 \times \max(x_1, x_2, \ldots, x_n)$$

This means the algorithm waits until the rolling mean has dropped to 0.9% of the maximum, at which point compilation activity has effectively subsided. We observed that in the steady-state phase, compilation activity typically falls to less than 1–2% of its maximum. We therefore experimented with fractions in this range (0.5% to 1.5%) and found that 0.9% yielded quite some consistent cutoff points across different benchmarks. These values worked well for the measurements used in this thesis.

We would like to emphasize that there is no principled method for choosing these values. The parameters presented above are the result of informed experimentation on our data and represent our best estimate of reasonable defaults. When applying this approach to other benchmark data, the same process of manual evaluation should be repeated to find values that suit the specific data at hand.

2.2 Normalization

After steady-state detection, the majority of warmup artifacts have been removed and the remaining iteration times are relatively stable. However, the data still requires normalization before it can be used as input to the analysis pipeline. This section explains why and presents the normalization strategy used in this work.

2.2.1 Motivation

Even after the SSD, while we assume the code is now in its peak performance and is stable, we recall from Chapter 1 that runtime artifacts such as garbage collection and compiler de-optimizations can still produce occasional spikes during steady-state execution. Standard normalization methods such as min-max scaling

and z-score normalization are sensitive to such spikes. In our experiments, when we initially applied z-score normalization to the post-SSD data, the spikes dominated the normalized representation. The model used in the analysis learned to distinguish measurements based on their spike patterns rather than their actual performance characteristics. For instance, under one configuration[1], spikes in the z-score normalized data reached 7–8 standard deviations above the mean, while a performance shift between two consecutive versions occupied only 0.5 standard deviations. The pipeline was unable to reliably distinguish whether a flagged transition reflected a genuine performance change or was driven by the presence of spikes in the measurement.

Furthermore, the analysis pipeline is trained on data from many different benchmarks, which operate at very different absolute iteration times. For the pipeline to learn meaningful representations across benchmarks, the data must be brought to a common scale during training. At inference time, the same normalization must be applied for consistency. The normalization must therefore achieve two goals: *robustness* to occasional spikes, and *comparability* across benchmarks.

2.2.2 Log-Transform

To address the issues described above, we apply a natural logarithm to the iteration times before any further normalization. The log-transform has two key properties:

1. **Compression of proportional deviations.** A spike that doubles the iteration time from t to $2t$ becomes a constant offset in log space:

$$\log(2t) = \log(2) + \log(t) \approx 0.69 + \log(t)$$

   The offset $+0.69$ is the same whether t is 200 ms or 5 000 ms. This prevents outliers from dominating the statistics, directly addressing the problem observed in Section 2.2.1.

2. **Consistent representation of relative changes.** A 5% performance change is always $\log(1.05) \approx 0.05$ in log space, regardless of baseline. This makes data from different benchmarks comparable when training the analysis pipeline, and preserves cross-version comparability at inference time.

Even in the absence of spikes, the log-transform is harmless — it is a monotonic transformation that preserves the ordering and relative structure of the data.

2.2.3 Robust Scaling with Median and IQR

After the log-transform, data from different configurations still operates at different absolute scales. To bring all configurations to a common range, we center

[1]Configuration C = (`batik-small`, 13, 12, 34), corresponding to the `batik-small` benchmark on machine host 13, platform type `graal-ee-master-jdk-11`, and GC configuration `G1 Garbage Collector` with 12 GB heap.

and scale using the median $\tilde{\mu}$ and interquartile range $\Delta = Q_{75} - Q_{25}$, computed over the concatenation of all log-transformed iteration times in a configuration C:

$$\hat{y} = \frac{y - \tilde{\mu}}{\Delta}$$

We use the median and IQR rather than the mean and standard deviation because they are robust to outliers: the median is unaffected by extreme values, and the IQR is determined only by the central 50% of the data. When the IQR is near zero (e.g., nearly constant iteration times), the implementation falls back to $\Delta = 1$.

2.3 Per-Configuration Preprocessing Pipeline

To summarize, the full preprocessing pipeline applied to each configuration C before the data enters the analysis pipeline is as follows:

1. **Steady-state detection.** For each measurement m_i in the configuration time series S_C, the CAS analysis (Section 2.1.1) identifies the cutoff index i^*. All iterations before i^* are discarded. Measurements where no cutoff is found are excluded.

2. **Log-transform.** The remaining iteration times are transformed by the natural logarithm (Section 2.2.2).

3. **Robust scaling.** The median $\tilde{\mu}$ and IQR Δ are computed over the concatenation of all log-transformed iteration times across all measurements in C. Each value is then centered and scaled as $\hat{y} = (y - \tilde{\mu})/\Delta$ (Section 2.2.3).

4. **Padding.** For our analysis pipeline, all measurements must have the same length. More on this in Chapter 4. To achieve this, the pre-processed iteration time sequences are padded with a const pad value to a fixed variable length.

Note that the statistics used for robust scaling (median and IQR) are computed *per configuration*, not globally or per individual measurement. This is because measurements within a configuration share the same benchmark, machine, and runtime settings, so their iteration times are on the same scale. Pooling the statistics across all measurements in C provides a more stable estimate than computing them per measurement, while still respecting the fact that different configurations operate at different scales.

After preprocessing, the resulting normalized sequences for each configuration are ready for analysis.

3 Related Work

Before presenting the analysis approach developed in this thesis, we briefly survey existing methods for detecting and analyzing performance changes in software systems. The methods discussed in this chapter are not directly used in our work, but they provide context for the design decisions made in later chapters and illustrate the landscape of techniques available for this class of problems.

3.1 Statistical Approaches

3.1.1 Changepoint Detection

3.1.2 Hypothesis Testing

3.2 Machine Learning Approaches

3.2.1 Supervised Methods

3.2.2 Unsupervised Methods

3.3 Summary

4 Analysis Approach and TS2Vec

The preprocessing pipeline described in Chapter 2 produces, for each configuration C, a set of preprocessed time series, one per benchmark measurement. The task that remains is to detect whether a performance change has occurred between consecutive compiler versions within C. This chapter presents the analysis strategy developed to address this task.

The central idea is to transform each measurement's time series into a fixed-length vector representation, an *embedding*, such that measurements with similar performance characteristics map to nearby points in the embedding space, and measurements that differ map to distant points. Under this formulation, detecting a performance change reduces to identifying transitions in the embedding space where consecutive compiler versions produce substantially different vectors.

The chapter is organized as follows. Section 4.1 formalizes this idea. Section 4.2 describes the representation strategies we explored before arriving at the final approach. Sections 4.3–4.3.4 present TS2Vec, the contrastive learning framework we adopted, including its architecture, loss function, and training procedure. Finally, Section 4.4 describes how we validated that the learned representations are meaningful.

4.1 Measurements to Vectors

Recall from Chapter 1 that a configuration time series S_C consists of benchmark measurements $(m_1, v_1), (m_2, v_2), \ldots, (m_N, v_N)$ ordered by compiler version. After preprocessing, each measurement m_i is a preprocessed time series $\mathbf{x}_i \in \mathbb{R}^T$, where T is the padded sequence length.

We seek a function that maps each measurement to a compact vector representation.

Definition. *The embedding function $f_\theta : \mathbb{R}^T \to \mathbb{R}^K$ is a parameterized function that maps each preprocessed measurement $\mathbf{x}_i$ to a K-dimensional embedding vector:*

$$\mathbf{z}_i = f_\theta(\mathbf{x}_i)$$

The embedding function should satisfy two properties[1]:

1. **Consistency.** If two measurements m_i and m_j exhibit the same performance behavior, their embeddings $\mathbf{z}_i$ and $\mathbf{z}_j$ should be close in the embedding space.

2. **Sensitivity.** If a genuine performance change occurs between m_i and m_{i+1}, the distance $\|\mathbf{z}_i - \mathbf{z}_{i+1}\|$ should be significantly larger than the typical distance between consecutive measurements under stable performance.

[1]The notions of "close" and "significantly larger" are intentionally left ambigious here. Their precise interpretation depends on the distance metric and the thresholding strategy used in the downstream analysis pipeline (Chapter 5).

The challenge lies in learning f_θ without labeled examples of performance changes. The measurements in the dataset are not annotated with ground-truth labels indicating whether a change has occurred. This rules out supervised approaches and motivates the use of self-supervised representation learning, where the encoder learns useful structure directly from the data.

4.2 Representation Strategies

Before arriving at the approach presented in the following sections, we explored several strategies for constructing the embedding function f_θ. This section briefly describes the alternatives considered and the limitations that led us to move beyond them.

4.2.1 Statistical Feature Extraction

The most straightforward approach is to define f as a hand-crafted function that extracts summary statistics from each time series. For each measurement $\mathbf{x}_i$, one could compute a feature vector from descriptive statistics of the iteration times, producing a fixed-length representation without any learning. This is where the idea of summarizing a benchmark measurement into a single vector originated.

However, this approach is fundamentally limited. Benchmark execution is governed by complex runtime dynamics and reducing a full execution trace to a handful of aggregate statistics cannot capture these interactions. Moreover, it discards the temporal structure of the time series entirely: two measurements with identical summary statistics but different temporal patterns, for example a gradual drift versus a sudden shift, would produce the same feature vector. For our task, where the *shape* of the execution profile carries information about the nature of a performance change, this loss of structure makes the approach unsuitable.

4.2.2 LSTM Autoencoders

A more principled approach is to *learn* the representation directly from the data using an autoencoder. We implemented an LSTM autoencoder in which an encoder network reads the input time series and compresses it into a latent vector $\mathbf{z} \in \mathbb{R}^d$, and a decoder network attempts to reconstruct the original sequence from $\mathbf{z}$ alone. The model is trained by minimizing the reconstruction error (mean squared error) between the input and its reconstruction. After training, the encoder's latent vector serves as the embedding.

The idea is appealing: if the model can reconstruct the full time series from a small latent vector, then that vector must contain the essential information about the measurement. In practice, however, the reconstruction objective itself turned out to be the problem. Minimizing reconstruction error (MSE) does not require the model to capture what *matters* for distinguishing measurements; it only requires the output to be close to the input on average. The decoder exploited this by learning to output the mean value of each sequence at every timestep. This trivially achieves low reconstruction error without encoding any temporal

structure. The resulting latent vectors were therefore uninformative, as they captured little beyond the global mean of each sequence.

Training was also computationally expensive due to the sequential nature of LSTM processing over long sequences, which made experimentation slow and impractical.

4.2.3 From Reconstruction to Contrastive Learning

The limitations of the previous approaches motivated a change in strategy. Rather than reconstructing the input or relying on manually crafted features, we turned to *contrastive learning*, a self-supervised paradigm in which the model learns representations by comparing inputs directly. The model is trained to place similar time series close together in the embedding space and dissimilar ones far apart, which aligns directly with the consistency and sensitivity properties defined in Section 4.1.

Among contrastive learning frameworks for time series, we adopted TS2Vec (7), which introduces a hierarchical contrastive objective that captures patterns at different levels of temporal detail.

4.3 TS2Vec

TS2Vec (7) is a contrastive learning framework for learning universal representations of time series, with demonstrated state-of-the-art results on time series classification, forecasting, and anomaly detection tasks. This section presents the framework in detail: the contrastive learning paradigm it builds on, the encoder architecture, the hierarchical loss function, and the training procedure.

4.3.1 The Contrastive Learning Objective

Consider a set of benchmark measurements. Some of them come from the same benchmark and exhibit similar execution patterns. Others come from different benchmarks and look different. Contrastive learning trains the encoder by presenting it with a task: given a query measurement, identify which other measurement is its "match" from a pool of candidates.

To formalize this, we define two types of pairs:

- **Positive pairs:** two representations that *should* be similar, because they originate from related inputs (e.g., two augmented views of the same time series).
- **Negative pairs:** two representations that *should not* be similar, because they originate from unrelated inputs (e.g., different time series in the same batch).

The encoder is trained to produce representations where positive pairs have high similarity and negative pairs have low similarity. This objective is expressed by the InfoNCE loss (Noise-Contrastive Estimation), introduced by Oord et al. [8].

Definition. *Let $\mathbf{r} \in \mathbb{R}^K$ be the representation vector of a query input produced by the encoder. Let $\mathbf{r}^+$ be the representation of its positive match, and let $\{\mathbf{r}_j^-\}_{j=1}^N$ be the representations of N negative samples. The InfoNCE contrastive loss is defined as:*

$$\mathcal{L} = -\log \frac{\exp(\mathbf{r} \cdot \mathbf{r}^+)}{\exp(\mathbf{r} \cdot \mathbf{r}^+) + \sum_{j=1}^{N} \exp(\mathbf{r} \cdot \mathbf{r}_j^-)}$$

The loss is composed of three components:

- **Dot product** ($\mathbf{r} \cdot \mathbf{r}^+$): measures the similarity between two representation vectors. The higher the dot product, the more similar the representations.
- **Softmax** (exp and the fraction): the exp function converts the similarity scores into a probability distribution over all $N + 1$ candidates (the positive and the N negatives). The fraction represents the probability that the model assigns to the correct match.
- **Cross-entropy** ($-\log$): converts the probability into a loss value. When the model confidently identifies the positive pair, the probability is close to 1 and the loss is close to 0. When it fails, the probability is low and the loss is high.

The key design choice in any contrastive framework is how positive and negative pairs are defined. TS2Vec introduces a strategy called *contextual consistency*: the representations of the same timestamp in two differently augmented views of the same time series form a positive pair. A context is generated by applying timestamp masking and random cropping to the input (described in Section 4.3.4). This strategy avoids assumptions about the data that other approaches make (9, 10) and only requires that the same point in time, viewed in two different contexts, should produce the same representation.

4.3.2 Encoder Architecture

The encoder f_θ maps an input time series $\mathbf{X} \in \mathbb{R}^{T \times F}$ to a per-timestamp representation $\mathbf{R} \in \mathbb{R}^{T \times K}$, where F is the number of input features per timestamp and K is the representation dimension. The encoder consists of three components: an input projection layer, a timestamp masking module, and a stack of dilated convolutional blocks. A final output projection layer maps the encoder's output to the representation dimension K.

Input projection. The raw input values are first lifted into a higher-dimensional space where the convolutional layers can operate. A fully connected layer maps each observation $\mathbf{x}_t \in \mathbb{R}^F$ to a latent vector $\mathbf{h}_t \in \mathbb{R}^H$, where $H > F$ is the hidden dimension of the encoder:

$$\mathbf{h}_t = W\mathbf{x}_t + \mathbf{b}$$

Here $W \in \mathbb{R}^{H \times F}$ is a learnable weight matrix and $\mathbf{b} \in \mathbb{R}^H$ is a learnable bias vector. This projection expands the dimensionality from F to H, providing the subsequent convolutional layers with a richer representation to operate on.

Timestamp masking. During training, a binary mask $\mathbf{m} \in \{0,1\}^T$ is sampled independently at each timestamp from a Bernoulli distribution with $p = 0.5$. Masked positions are set to zero in the latent space: $\tilde{\mathbf{h}}_t = m_t \cdot \mathbf{h}_t$. The mask is resampled for each forward pass, so the same input produces different partially visible sequences each time. This forces the encoder to learn representations from incomplete context. Masking is applied to the latent vectors (after the input projection) rather than to the raw input values. This is because setting a raw input to zero would be indistinguishable from an actual iteration time of zero, whereas in the projected H-dimensional space, the zero vector does not correspond to any real input. The role of timestamp masking in generating augmented views for the contrastive loss is described in Section 4.3.3.

Dilated convolutional blocks. After the input projection, each timestamp's H-dimensional vector contains information about that timestamp only. The role of the convolutional blocks is to incorporate contextual information from surrounding timestamps into each representation.

A 1-D convolution applies a *filter* (or *kernel*), a small vector of k learnable weights, at each position t in the sequence. The filter computes a weighted sum over k neighboring positions and produces a single output value. In a standard convolution, these k positions are consecutive. The key idea in this architecture is *dilation*: instead of consecutive positions, the filter operates on k positions with a fixed spacing d between them. A convolution with dilation d at position t operates on positions $[t-d,\ t,\ t+d]$, allowing each layer to capture a wider context without increasing the number of parameters.

The encoder stacks L residual blocks with exponentially increasing dilation factors $(2^0, 2^1, \ldots, 2^{L-1})$, where the l-th block uses a dilation of 2^l. Each block doubles the spacing but always reads exactly k positions. Figure 4.1 illustrates this for the first three blocks.

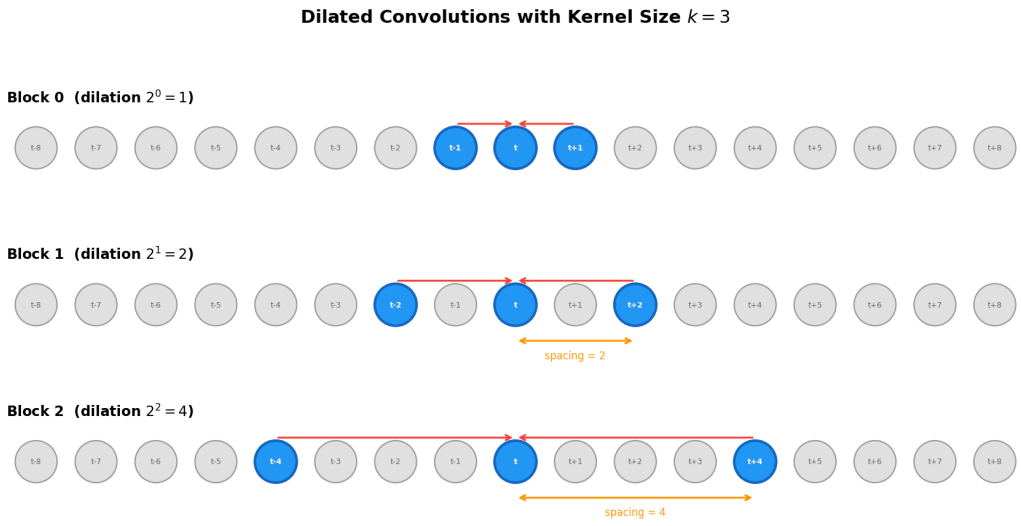

Figure 4.1 Dilated convolutions at increasing dilation factors. Blue circles indicate the positions read by the filter at each block. The spacing between positions doubles with each block while the number of read positions remains fixed at k.

Because the blocks are stacked, the context compounds: block 1 does not

operate on the raw input but on the output of block 0, where each position already carries information from its immediate neighbors. As a result, the total context accessible to each timestamp, known as the *receptive field*, grows exponentially with the number of blocks.

Residual block formulation. Let $\mathbf{h}^{(l-1)}$ denote the sequence of H-dimensional vectors entering the l-th block (i.e., the output of the previous block, or the masked projection for $l = 0$). Each block applies two dilated convolutions and produces the updated sequence $\mathbf{h}^{(l)}$:

$$\mathbf{h}^{(l)} = \mathrm{ReLU}\Big(\mathrm{Conv}_{2^l}\Big(\mathrm{ReLU}\Big(\mathrm{Conv}_{2^l}(\mathbf{h}^{(l-1)})\Big)\Big)\Big) + \mathbf{h}^{(l-1)}$$

The input $\mathbf{h}^{(l-1)}$ passes through two dilated convolutions, each followed by a ReLU activation. The ReLU function, $\mathrm{ReLU}(x) = \max(0, x)$, introduces non-linearity: without it, stacking multiple convolutions would collapse into a single linear operation.

The term $+\,\mathbf{h}^{(l-1)}$ is the *residual connection*: it adds the block's original input directly to its output. In other words, the block does not learn the full output representation from scratch. Instead, it only learns *what to add* to the input that is already there. If the block has nothing useful to contribute, it can output values close to zero and the input passes through unchanged. This keeps training stable even as the network grows deeper.

Output projection. After all L convolutional blocks, a final linear layer maps the output of the last block $\mathbf{h}_t^{(L)} \in \mathbb{R}^H$ to the representation dimension K:

$$\mathbf{r}_t = W_{\text{out}}\mathbf{h}_t^{(L)} + \mathbf{b}_{\text{out}}$$

where $W_{\text{out}} \in \mathbb{R}^{K \times H}$ and $\mathbf{b}_{\text{out}} \in \mathbb{R}^K$ are learnable parameters. This produces the per-timestamp representation vectors $\mathbf{r}_t \in \mathbb{R}^K$. The output of the encoder is therefore a matrix $\mathbf{R} \in \mathbb{R}^{T \times K}$: one K-dimensional vector for each of the T timestamps. During training, these per-timestamp representations are passed directly into the contrastive loss (Section 4.3.3), which operates on individual timestamps. No aggregation is performed at this stage.

All learnable parameters in the encoder, including the input projection (W, $\mathbf{b}$), the convolution filter, and the output projection (W_{out}, $\mathbf{b}_{\text{out}}$), are part of the encoder parameters θ and are trained jointly.

4.3.3 Hierarchical Contrastive Loss

Section 4.3.1 introduced the general InfoNCE loss with a single positive pair and a set of negatives. In TS2Vec, this idea is extended by defining two losses, *temporal* and *instance-wise*, which differ in how they construct their negative pairs. Both losses operate on two sets of per-timestamp representations, $\mathbf{R}$ and $\mathbf{R}'$. These are obtained through the following process:

1. Two overlapping segments are randomly cropped from the original time series, producing two partial views of the same measurement (Figure 4.2).

2. Each cropped view is passed through the encoder, where timestamp masking is applied internally (Section 4.3.2).

3. The encoder outputs a matrix of K-dimensional per-timestamp representations for each view: $\mathbf{R}$ from view 1 and $\mathbf{R}'$ from view 2.

Since the two crops overlap only partially and the masks differ, the encoder sees different partial perspectives of the same underlying measurement. The losses defined below are computed only on the timestamps within the overlap region, denoted Ω.

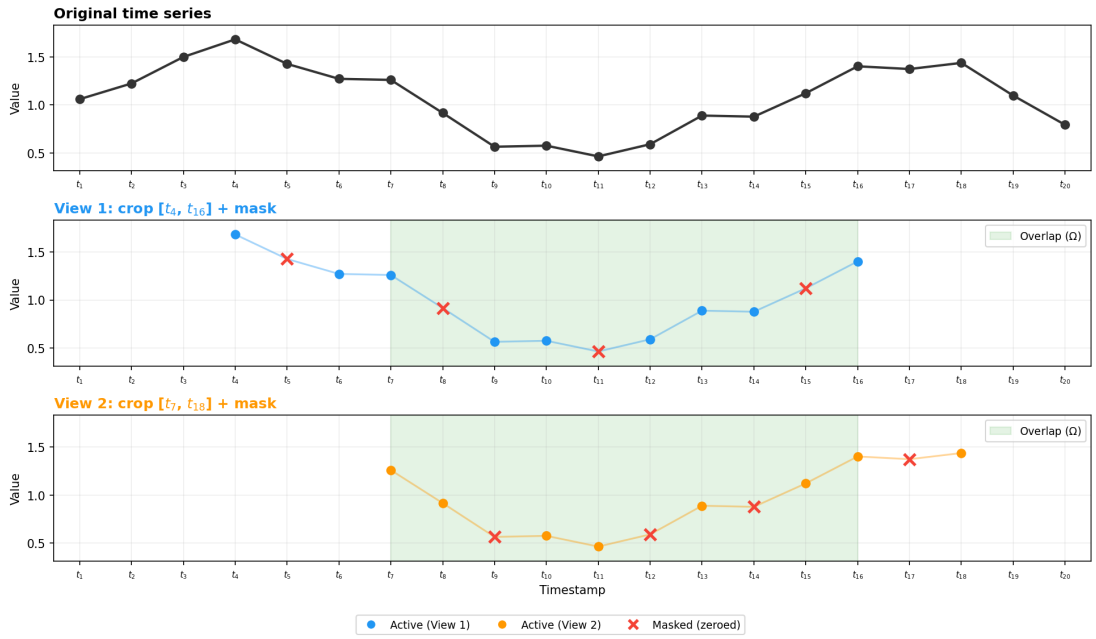

Figure 4.2 Example of two augmented views generated from the same time series. The original sequence (top) is cropped into two overlapping segments (middle and bottom), and timestamps are independently masked (red crosses) in each view. The green shaded region indicates the overlap Ω.

Temporal contrastive loss. The idea behind this loss is that each timestamp in a time series should be distinguishable from every other timestamp in the same series. If the encoder produces the same representation for timestamp 10 and timestamp 200, it has failed to capture how the execution evolves over time.

Definition. *For a given time series i, let $\mathbf{r}_{i,t}$ and $\mathbf{r}'_{i,t}$ denote the representations at timestamp t from two augmented views. The temporal contrastive loss at position (i,t) is:*

$$\ell_{\text{temp}}^{(i,t)} = -\log \frac{\exp(\mathbf{r}_{i,t} \cdot \mathbf{r}'_{i,t})}{\sum_{t' \in \Omega} \Big(\exp(\mathbf{r}_{i,t} \cdot \mathbf{r}'_{i,t'}) + \mathbf{1}_{[t \neq t']} \exp(\mathbf{r}_{i,t} \cdot \mathbf{r}_{i,t'}) \Big)}$$

where Ω is the set of timestamps for which both augmented views have valid representations (since the two views are random crops that only partially overlap, the loss is computed on their shared region only).

The positive pair is $(\mathbf{r}_{i,t}, \mathbf{r}'_{i,t})$: the same timestamp t in two augmented views of the same time series i. The negatives are all other timestamps $t' \neq t$ from the same time series. The term $\mathbf{1}_{[t \neq t']}$ is the indicator function: it equals 1 when $t \neq t'$ and 0 otherwise, preventing the positive pair from appearing among its own negatives.

Instance-wise contrastive loss. While the temporal loss teaches the encoder to distinguish *when* something happens within a single time series, the instance-wise loss teaches it to distinguish *which* time series it is looking at. Here, an *instance* refers to a single time series in the training batch. At any given timestamp, the representation of one time series should be distinguishable from the representations of all other time series in the same batch.

Definition. *For a fixed timestamp t, the instance-wise contrastive loss at position (i, t) is:*

$$\ell_{\text{inst}}^{(i,t)} = -\log \frac{\exp(\mathbf{r}_{i,t} \cdot \mathbf{r}'_{i,t})}{\sum_{j=1}^{B} \left(\exp(\mathbf{r}_{i,t} \cdot \mathbf{r}'_{j,t}) + \mathbf{1}_{[i \neq j]} \exp(\mathbf{r}_{i,t} \cdot \mathbf{r}_{j,t}) \right)}$$

where B is the number of time series in the batch.

The positive pair is again $(\mathbf{r}_{i,t}, \mathbf{r}'_{i,t})$: the same time series, same timestamp, different view. The negatives are now representations from *other time series $j \neq i$* at the same timestamp t.

Combined dual loss. Before combining the two losses, we note the following observation.

Observation. *The temporal and instance-wise losses are complementary: the temporal loss captures how a time series evolves over time, while the instance-wise loss captures what makes one time series different from another.*

Since the two losses address different aspects of the representation, they can be combined into a single dual loss:

Definition. *The dual contrastive loss over all time series and timestamps is:*

$$\mathcal{L}_{\text{dual}} = \frac{1}{NT} \sum_{i} \sum_{t} \left(\ell_{\text{temp}}^{(i,t)} + \ell_{\text{inst}}^{(i,t)} \right)$$

where N is the number of time series in the batch and T is the number of timestamps.

Hierarchical application. The dual loss defined above operates on individual per-timestamp representations. However, patterns in a time series may span different lengths: some affect only a few timestamps, while others extend over large segments. Computing the loss at a single level of detail may not be sufficient to capture all of them. To address this, TS2Vec computes the dual loss at multiple levels of granularity.

After computing $\mathcal{L}_{\text{dual}}$ on the original per-timestamp representations, the representations are compressed by max-pooling pairs of consecutive timestamps

into one, halving the time dimension. The dual loss is computed again on these compressed representations. This process repeats, halving the time dimension each time, until only a single representation remains per time series.

Definition. *The hierarchical contrastive loss is the average of the dual loss computed at each level of granularity:*

$$\mathcal{L}_{\text{hier}} = \frac{1}{D} \sum_{d=1}^{D} \mathcal{L}_{\text{dual}}^{(d)}$$

where D is the total number of levels and $\mathcal{L}_{dual}^{(d)}$ is the dual loss computed after $d-1$ pooling operations.

Algorithm 1 Hierarchical contrastive loss

procedure HierLoss($\mathbf{R}, \mathbf{R}'$)
  $\mathcal{L}_{\text{hier}} \leftarrow \mathcal{L}_{\text{dual}}(\mathbf{R}, \mathbf{R}')$
  $d \leftarrow 1$
  while time_length($\mathbf{R}$) > 1 **do**
    $\mathbf{R} \leftarrow \text{MaxPool1d}(\mathbf{R},\ \text{kernel\_size} = 2)$
    $\mathbf{R}' \leftarrow \text{MaxPool1d}(\mathbf{R}',\ \text{kernel\_size} = 2)$
    $\mathcal{L}_{\text{hier}} \leftarrow \mathcal{L}_{\text{hier}} + \mathcal{L}_{\text{dual}}(\mathbf{R}, \mathbf{R}')$
    $d \leftarrow d + 1$
  end while
  $\mathcal{L}_{\text{hier}} \leftarrow \mathcal{L}_{\text{hier}} \,/\, d$
  return $\mathcal{L}_{\text{hier}}$
end procedure

At the first level ($d = 1$), the loss operates on individual timestamps. At each subsequent level, the time dimension is halved, so the representations summarize progressively larger segments of the time series. At the final level ($d = D$), a single representation remains per time series, and the loss operates at the level of the entire measurement. This hierarchical structure forces the encoder to produce representations that are meaningful whether examined at the level of a single iteration or an entire benchmark execution. The full procedure is summarized in Algorithm 1.

4.3.4 Training Procedure

Each training step proceeds as follows:

1. **Random cropping.** Two overlapping subsequences are sampled from the input time series. Let $[a_1, b_1]$ and $[a_2, b_2]$ be the two segments, with $a_1 \leq a_2 \leq b_1 \leq b_2$. The contrastive loss is computed only on the overlapping region $[a_2, b_1]$, where both views have valid data.

2. **Encoding.** Each crop is passed through the shared encoder f_θ, which applies timestamp masking internally (Section 4.3.2), producing per-timestamp representations $\mathbf{R}$ and $\mathbf{R}'$.

3. **Loss computation.** The hierarchical contrastive loss $\mathcal{L}_{\text{hier}}$ is computed over the overlapping region of the two views (Algorithm 1).

4. **Optimization.** The encoder parameters θ are updated via gradient descent using the AdamW optimizer with weight decay regularization.

The full training procedure is summarized in Algorithm 2.

Algorithm 2 TS2Vec training procedure

procedure FIT($\mathcal{X}, f_\theta, \alpha, E$)
  for epoch $= 1$ to E **do**
    for each mini-batch $\{\mathbf{x}_1, \ldots, \mathbf{x}_B\} \subset \mathcal{X}$ **do**
      $\mathcal{R} \leftarrow \{\}, \quad \mathcal{R}' \leftarrow \{\}$
      for $i = 1$ to B **do**
        Sample two overlapping crops from $\mathbf{x}_i$
        $\mathcal{R}_i \leftarrow f_\theta(\text{crop}_{i,1})$
        $\mathcal{R}'_i \leftarrow f_\theta(\text{crop}_{i,2})$
      end for
      $\mathcal{L}_{\text{hier}} \leftarrow \text{HIERLOSS}(\mathcal{R}, \mathcal{R}')$
      $\theta \leftarrow \theta - \alpha \nabla_\theta \mathcal{L}_{\text{hier}}$
    end for
  end for
end procedure

Note that both random cropping and timestamp masking are applied only during training. At inference time, no augmentation is performed: the full unmasked sequence is passed through the encoder, producing per-timestamp representations $\mathbf{R} \in \mathbb{R}^{T \times K}$. To obtain a single embedding vector per measurement, we apply max-pooling over the time dimension:

$$\mathbf{z} = \max_{t=1}^{T} \mathbf{r}_t$$

This takes the element-wise maximum across all T timestamp representations, producing a fixed-length vector $\mathbf{z} \in \mathbb{R}^K$ regardless of sequence length. This vector $\mathbf{z}$ is the final embedding used in the anomaly detection pipeline.

4.4 Encoder Training and Validation

The previous sections presented TS2Vec as a general framework. This section describes how we trained the encoder on our benchmark data and how we validated that the resulting representations are meaningful. Validating the encoder before deploying it in the anomaly detection pipeline is essential: if the representations do not capture the relevant structure of the data, no downstream analysis can compensate. The analysis in this section provides initial evidence that the encoder produces useful representations. However, the ultimate validation is its performance in the anomaly detection pipeline, where the quality of the embeddings directly determines the ability to detect performance changes between compiler versions. This is the subject of Chapter 5.

4.4.1 Training Configuration

The encoder was trained on approximately 60,000 preprocessed benchmark measurements spanning data from 2016 to 2020, as described in Chapter 1. Training ran for 37 epochs. Table 4.1 lists the hyperparameters used.

Parameter	Value
Input dimension (F)	1
Hidden dimension (H)	128
Representation dimension (K)	320
Dilated convolutional blocks (L)	10
Kernel size (k)	3
Mask ratio (p)	0.5
Optimizer	AdamW (weight decay 10^{-4})
Learning rate	10^{-3} (cosine annealing)

Table 4.1 TS2Vec encoder hyperparameters.

Figure 4.3 shows the contrastive loss over the course of training. The loss decreases steadily and converges after approximately 30 epochs. Convergence indicates that the encoder has reached a state where positive pairs are consistently assigned high similarity and negative pairs low similarity.

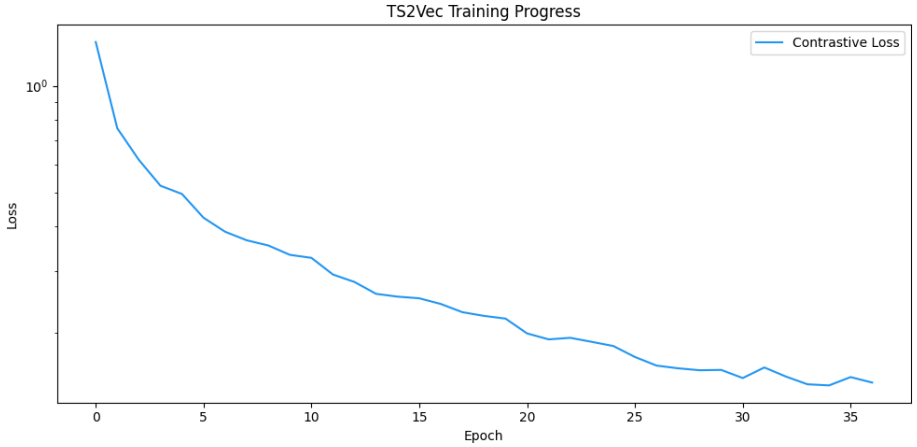

Figure 4.3 TS2Vec contrastive loss (log scale) over the course of training. The steady decrease indicates that the encoder is learning to produce discriminative representations.

4.4.2 t-SNE Visualization

To visualize the structure of the learned embedding space, we encoded a random sample of 5000 measurements with a minimum sequence length of 100 iterations (due to the limitations discussed in Section 4.4.4) and projected the resulting K-dimensional vectors to two dimensions using t-SNE (t-distributed Stochastic Neighbor Embedding).

Figure 4.4 shows the projection colored by sequence length. The embedding space is clearly structured: measurements do not form a uniform cloud but are organized into distinct regions. Measurements of similar length tend to occupy

the same areas, though this grouping is not purely driven by length, as we observe in the next figure.

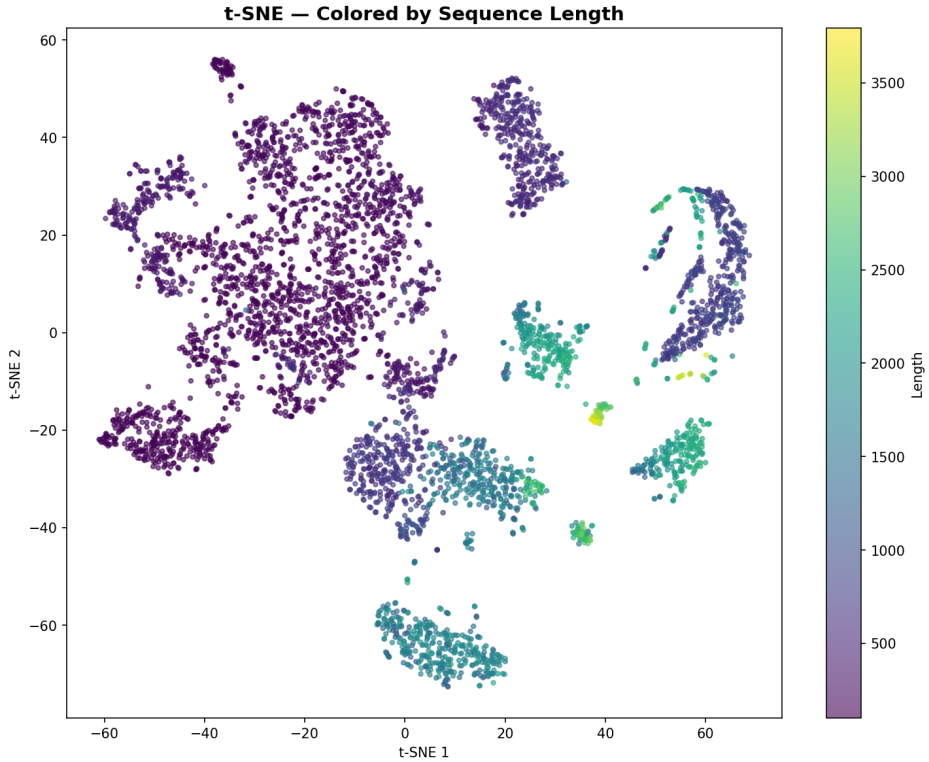

Figure 4.4 t-SNE projection of TS2Vec embeddings for 5000 measurements, colored by sequence length.

Figure 4.5 shows the same projection colored by benchmark name. Several benchmarks form tight, well-separated clusters, confirming that the encoder has learned to group measurements with similar execution characteristics. Other benchmarks, particularly in the denser region, show weaker but still visible grouping. Notably, some benchmarks of similar length cluster separately (e.g., `xalan-large` forms a distinct cluster despite sharing a similar length with nearby benchmarks), indicating that the encoder captures more than sequence length alone.

From a practical perspective, such visualizations can guide the performance engineer on which benchmarks are likely to produce reliable results in the anomaly detection pipeline: benchmarks that form tight clusters are well-captured by the encoder and can be analyzed with higher confidence, while those with weaker separation may require more careful interpretation.

4.4.3 Nearest-Neighbor Retrieval

A more direct test of representation quality is to check whether measurements that are close in the embedding space actually look similar when plotted. We selected 50 random query measurements and retrieved the five nearest neighbors for each by cosine distance in the embedding space. In the majority of cases, the neighbors exhibited similar temporal patterns to the query. Figure 4.6 shows one representative example.

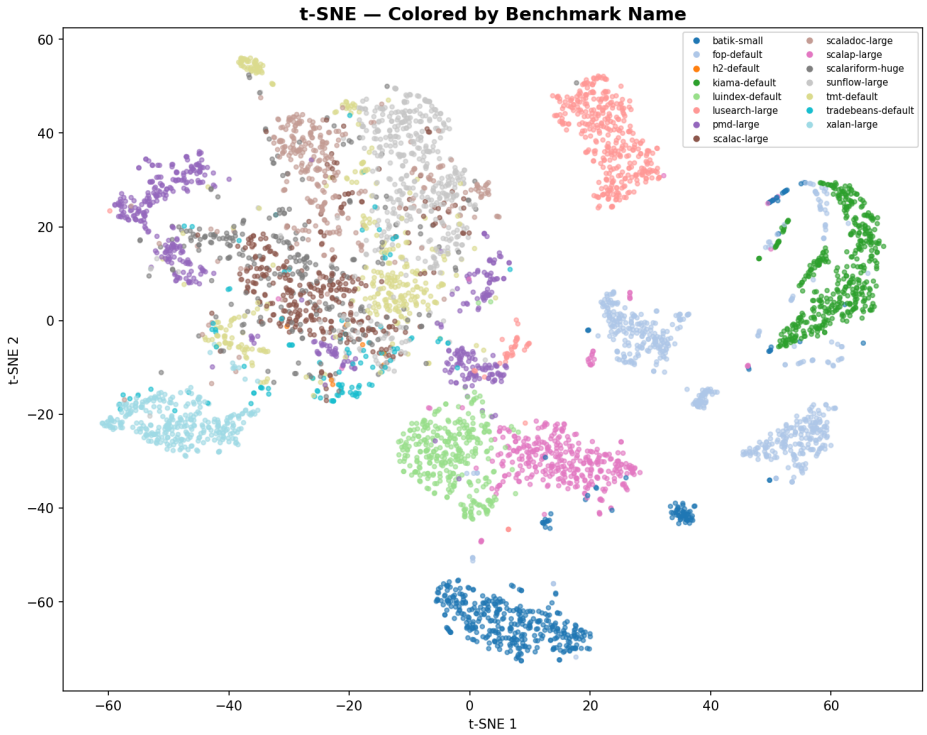

Figure 4.5 t-SNE projection of TS2Vec embeddings for 5000 measurements, colored by benchmark name. Several benchmarks form distinct clusters, while others show partial overlap.

The query and its neighbors share comparable amplitude, spike behavior, and overall shape, confirming that proximity in the embedding space corresponds to meaningful similarity in execution behavior.

4.4.4 Limitations

During validation, we observed that the encoder produces less reliable representations for measurements with short sequences. We believe this is due to two factors. First, shorter sequences provide the encoder with less temporal context to learn from, which may result in less discriminative embeddings. Second, the contrastive loss relies on having a sufficient number of timestamps to construct meaningful positive and negative pairs; with very short sequences, the overlap region after cropping contains few timestamps, which likely reduces the effectiveness of the temporal loss.

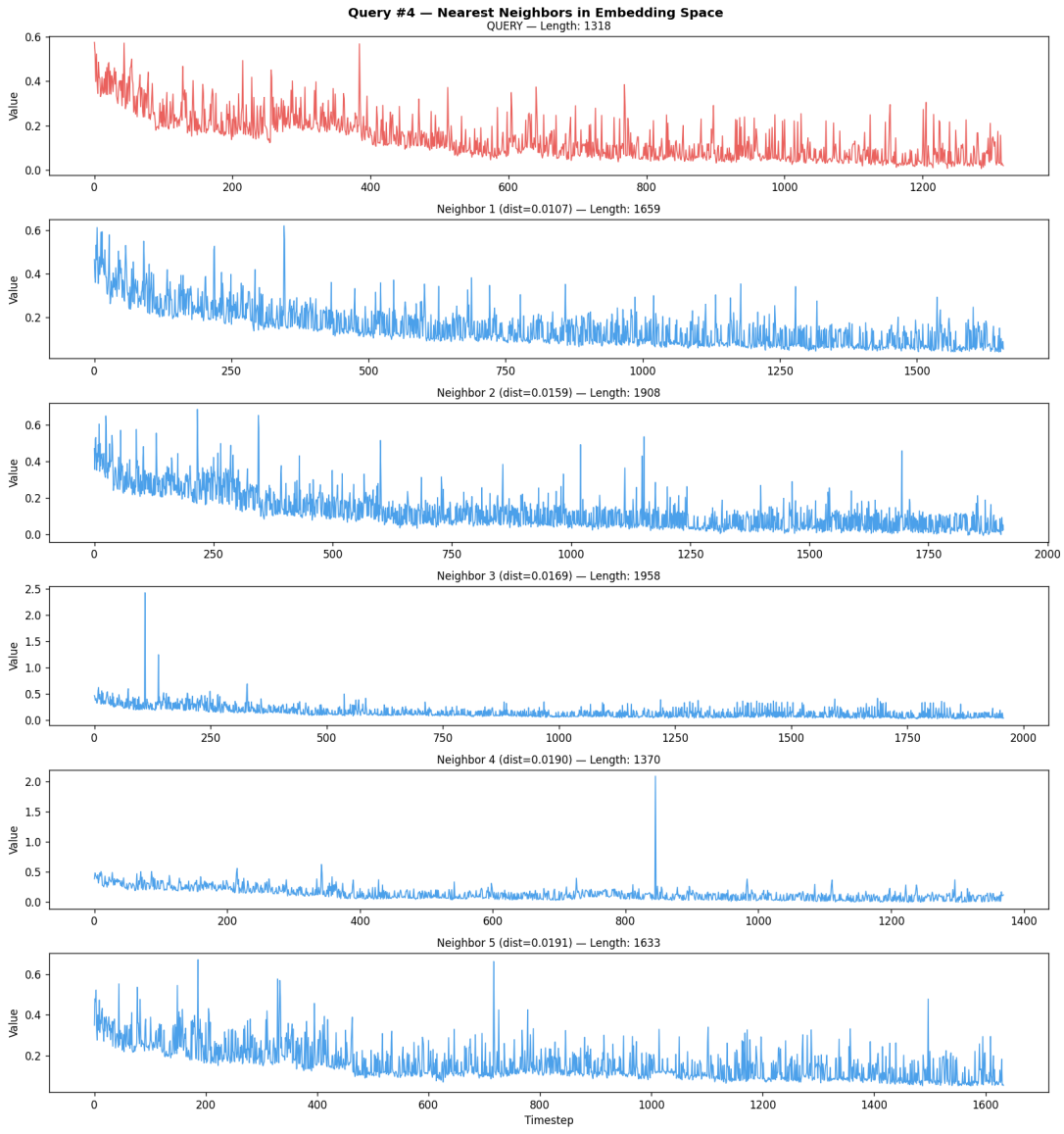

Figure 4.6 A query measurement (red) and its five nearest neighbors in the embedding space (blue), retrieved by cosine distance.

5 Anomaly Detection Pipeline and Reporting

The previous chapters described the preprocessing of raw benchmark data (Chapter 2) and the TS2Vec encoder that transforms each measurement into an embedding vector (Chapter 4). This chapter describes the full anomaly detection pipeline that brings these components together: how the dataset is accessed and organized, how the encoder is trained and applied to detect performance changes, and how the results are reported.

The pipeline is implemented as a sequence of independent scripts, each performing a well-defined step. All numerical parameters used throughout the pipeline (thresholds, hyperparameters, scoring criteria) are provided with reasonable defaults but can be overridden via a configuration file (`pipeline.conf`), allowing users to adjust the pipeline behavior without modifying the source code.

5.1 Pipeline Overview

The pipeline consists of the following steps:

1. **Index construction.** The raw dataset is scanned and an index mapping each configuration to its measurement directories is built and persisted as a pickle file. This step is run once per dataset.

2. **Training data preparation.** Using the index, all measurements are loaded, steady-state detection and per-configuration normalization are applied, and the resulting sequences are padded and saved as a single NumPy file for training.

3. **Encoder training.** The TS2Vec encoder is trained on the prepared data using the contrastive learning procedure described in Chapter 4.

4. **Configuration selection.** The index is scanned to identify configurations with sufficient post-SSD sequence length for reliable analysis.

5. **Per-configuration analysis.** For each selected configuration, the pipeline preprocesses, encodes, computes consecutive distances, flags anomalies, scores reliability, and generates reports.

6. **Cross-configuration corroboration.** Flagged transitions are compared across configurations to assess confidence.

5.2 Data Loading and Metadata Resolution

The first step of the pipeline constructs an index of all valid measurements in the dataset. The index builder expects a *base directory* containing a `measurement` directory (which stores all measurement directories in the triple-nested structure described in (1)) and a `metadata` directory (which contains the lookup files for

resolving numerical identifiers). Multiple base directories can be provided to combine data from different years.

As described in Section 1.3.1, each measurement occupies its own directory containing two files used by the pipeline: a `default.csv` file and a `metadata` JSON file.

1) `default.csv`. This file contains one row per benchmark iteration. The pipeline reads the following columns:

- `iteration_time_ns`: the wall-clock duration of each iteration in nanoseconds.
- `compilation_time_ms`: the aggregate compiler thread execution duration per iteration in milliseconds, used by the steady-state detector (Chapter 2).

2) `metadata`. This file contains numerical identifiers that the pipeline resolves into the four attributes of the configuration tuple $c(m) = (b, h, p, g)$ (Section 1.4.1) and the compiler version:

- **Benchmark name**: via `benchmark_workload`.
- **Machine host**: direct identifier.
- **Platform type, compiler version**: via `platform_installation`.
- **GC configuration**: direct identifier.

5.2.1 Filtering

The pipeline applies the following filters during index construction to retain only the measurements relevant to this work:

- **Platform types**: only JIT compilation platforms are retained, as this thesis focuses on JIT compilation (Section 1.2.1). All AOT (Native Image) platforms are excluded.
- **GC configurations**: only JIT-related garbage collection configurations are retained.
- **Machine hosts**: only measurements from known, dedicated machine hosts are included.

5.2.2 Deduplication

We recall that multiple measurements for the same configuration and compiler version can exist in the dataset (Section 1.4.3). To handle this, the pipeline retains only the measurement with the most recorded iterations, as it provides the richest data for analysis. After deduplication, each configuration contains at most one measurement per compiler version.

The resulting index is saved as a persistent file and used by all subsequent steps of the pipeline.

5.3 Training and Data Preparation

Before the encoder can be used for analysis, it must be trained. The training process consists of two steps: preparing the training data from the index, and running the training procedure described in Section 4.3.4.

The data preparation step loads all measurements from the index, applies steady-state detection and per-configuration normalization, pads all sequences to a common length, and saves the result as a single file. This file is then used as input to the training script, which creates and trains the TS2Vec encoder using the parameters specified in the configuration file. The trained model is saved for use in the analysis step.

5.4 Configuration Selection

Not all configurations in the index are suitable for analysis. As discussed in Section 4.4.4, the encoder produces less reliable representations for measurements with short sequences. To address this, the pipeline includes a configuration selection step that estimates the median post-SSD sequence length for each configuration (by sampling a subset of its measurements) and retains only those exceeding a minimum threshold. The selected configurations are saved as a JSON file used by the analysis step. Alternatively, users who are confident that their configurations of interest are suitable for analysis can provide their own configuration file directly, bypassing this selection step.

5.5 Per-Configuration Analysis

The pipeline processes each selected configuration independently. For a given configuration C, the measurements are preprocessed, encoded by the trained TS2Vec encoder, and ordered by compiler version.

5.5.1 Cosine Distance

For each pair of consecutive compiler versions (v_i, v_{i+1}), the cosine distance between their embedding vectors is computed:

$$d_i = 1 - \frac{\mathbf{z}_i \cdot \mathbf{z}_{i+1}}{\|\mathbf{z}_i\| \, \|\mathbf{z}_{i+1}\|}$$

Cosine distance measures the angular difference between two vectors, independent of their magnitude. Under stable performance, consecutive embeddings should be similar and the distances should be small.

5.5.2 Anomaly Flagging

A transition is flagged as a potential anomaly if its distance d_i satisfies two conditions:

1. **Percentile threshold.** The distance must exceed a high percentile P of all consecutive distances within the configuration:

$$d_i > P(d_1, d_2, \ldots, d_{N-1})$$

2. **Z-score threshold.** The distance must have a z-score of at least $z_{\min}$:

$$z_i = \frac{d_i - \mu_d}{\sigma_d} \geq z_{\min}$$

where μ_d and σ_d are the mean and standard deviation of the distance sequence.

The percentile threshold identifies the largest distances in the configuration. However, exceeding the percentile alone does not guarantee that a distance is meaningfully different from the rest. The z-score provides this guarantee: it measures how far a distance is from the mean relative to the spread of all distances. A transition is only flagged when both conditions are met, ensuring that the flagged distance is both among the largest and significantly separated from the typical baseline.

Both thresholds are configurable. In our case, the defaults were determined to deliberately represent a strict setting: we prefer to miss a borderline anomaly than to report a false positive. We note that the effectiveness of both thresholds depends on having a sufficient number of compiler versions in the configuration: with very few versions, the percentile has too few data points to be statistically meaningful and the z-score distribution is poorly estimated. In general, the more versions available, the more reliable the flagging.

5.5.3 Reliability Scoring

The pipeline assigns a reliability rating to each configuration based on four warning signals:

1. **High mean distance**: the baseline distance between consecutive versions is high, indicating noisy embeddings.
2. **High length range ratio**: the ratio between the longest and shortest sequence is large, suggesting SSD instability.
3. **Few compiler versions**: the percentile threshold is unreliable with very few data points.
4. **Low maximum z-score**: even the strongest flagged anomaly barely stands out from the noise.

- **STRONG** (0 warnings): results are trustworthy.
- **WEAK** (1 warning): interpret with caution.
- **RECOMMEND-SKIP** (2+ warnings): results may be unreliable.

5.6 Cross-Configuration Corroboration

A performance change caused by a compiler commit is likely to affect multiple configurations that share the same benchmark and platform type. The pipeline groups all flagged transitions by benchmark type, platform type, and version pair across all machine hosts and GC configurations, and assigns a confidence level:

- **STRONG**: flagged in a high proportion of relevant configurations.
- **MODERATE**: flagged in a moderate proportion with multiple independent detections.
- **WEAK**: flagged in multiple configurations but at a low proportion.
- **SINGLE-CONFIG**: flagged in only one configuration.

5.7 Reporting

For each configuration, the pipeline generates:

- A text report with preprocessing statistics, distance statistics, flagged anomalies, and reliability rating.
- A CSV of all consecutive distances with flagged status.
- A t-SNE visualization of the embeddings colored by version order.
- For each flagged anomaly, a plot showing the raw iteration times of the current and previous compiler versions, alongside their nearest neighbors in the embedding space. The neighbors are included to visually confirm that proximity in the embedding space corresponds to similar execution behavior.

Additionally, the pipeline produces a global summary containing the full results for every analyzed configuration, including the flagged version transitions with their distances, z-scores, thresholds, and reliability ratings. A cross-configuration report listing corroborated transitions with their confidence levels is also generated. The version transitions in the cross-configuration report can be mapped back to the summary to obtain the full details for the corresponding anomaly.

5.8 Example

To illustrate the pipeline output, we present a concrete example from the configuration `batik-small` on machine host 13, platform type 12, GC configuration 34. This configuration contains 61 compiler versions and received a STRONG reliability rating.

Figure 5.1 shows an excerpt from the generated report for this configuration.

Figure 5.2 shows the anomaly plot for the transition from version 55313 to version 55422. The flagged version (red) exhibits noticeably higher iteration times compared to the previous version (orange) and the nearest neighbors in the embedding space (blue). The neighbors confirm that the previous version's

```
============================================================
CONFIG ANALYSIS REPORT
============================================================

Benchmark:     batik-small
Machine Host:  13
Platform Type: 12
GC Config:     34

--- Data ---
Raw entries in index:   61
Valid after SSD:        61
Sequence length (max):  2047
Sequence length (min):  1103

--- Consecutive Distances ---
N transitions:          60
Mean distance:          0.151316
Threshold:              0.469800
Flagged anomalies:      2

--- Reliability ---
Rating:                 STRONG

--- Flagged Anomalies ---
  v55313 -> v55422: dist=0.502905, z=2.49
  v55750 -> v55767: dist=0.489845, z=2.40
============================================================
```

Figure 5.1 Excerpt from the generated report for a STRONG-rated configuration. Two version transitions were flagged, both with distances well above the threshold and z-scores exceeding 2.0.

execution profile is representative of normal behavior for this benchmark, and that the flagged version deviates from it.

This transition was also flagged across 7 out of 16 configurations sharing the same benchmark and platform type, receiving a STRONG cross-configuration confidence rating. This corroboration across multiple machine hosts and GC configurations provides strong evidence that the performance change is attributable to the compiler commit rather than environmental factors.

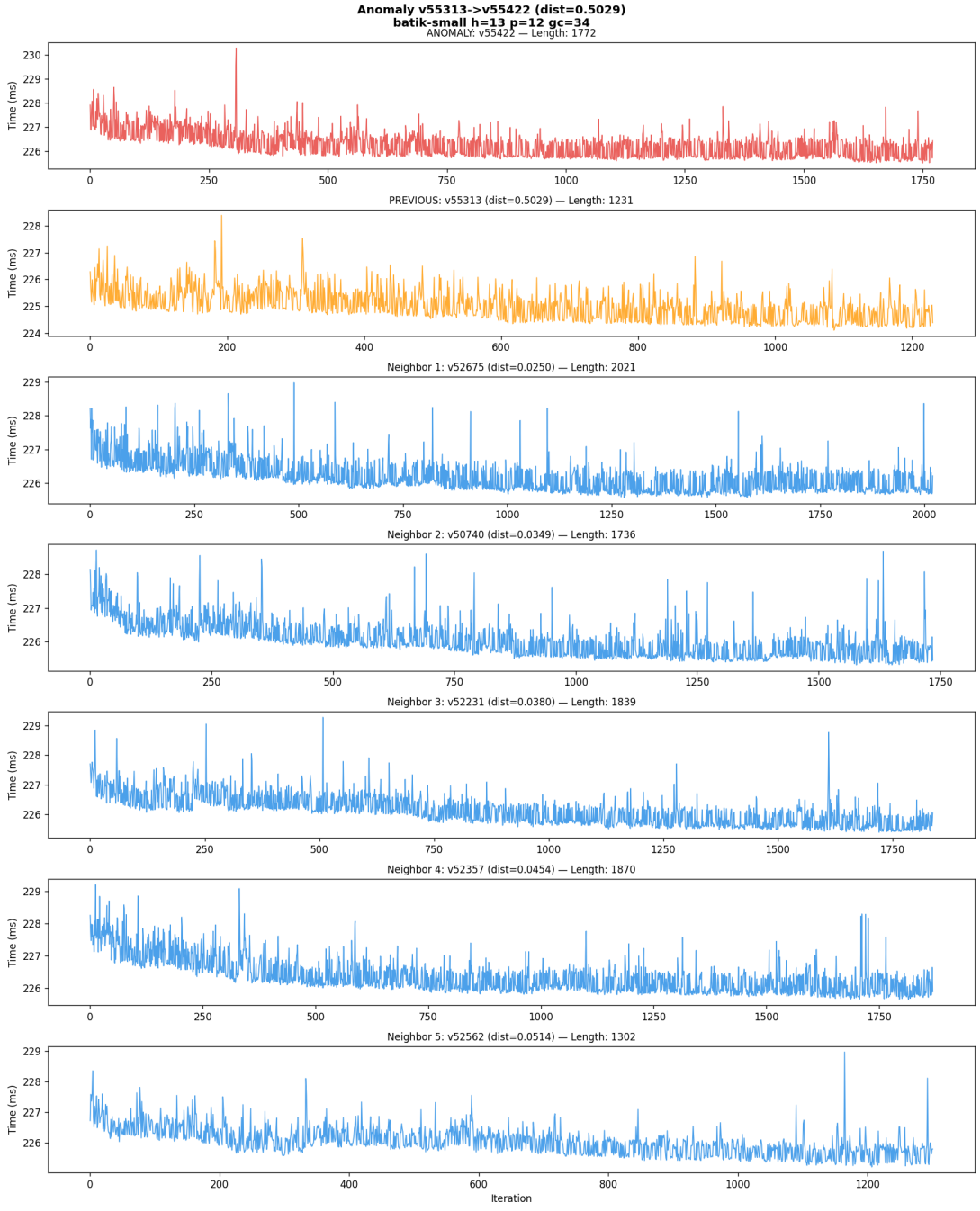

Figure 5.2 Anomaly plot for the transition v55313→v55422 in the `batik-small` configuration. The flagged version (red) shows higher iteration times than the previous version (orange) and the nearest neighbors (blue).

Conclusion

Bibliography

1. Bulej, Lubomír; Horký, Vojtěch; Tucci, Michele; Tůma, Petr; Farquet, François; Leopoldseder, David; Prokopec, Aleksandar. GraalVM Compiler Benchmark Results Dataset (Data Artifact). In: *Companion of the 2023 ACM/SPEC International Conference on Performance Engineering (ICPE '23 Companion)*. New York, NY, USA: ACM, 2023, pp. 65–69. Available from doi: 10.1145/3578245.3585025.

2. Hartmann, Tobias; Noll, Albert; Gross, Thomas. Efficient Code Management for Dynamic Multi-Tiered Compilation Systems. In: *Proceedings of PPPJ 2014*. 2014. Available from doi: 10.1145/2647508.2647513.

3. Traini, Luca; Cortellessa, Vittorio; Di Pompeo, Daniele; Tucci, Michele. Towards Effective Assessment of Steady State Performance in Java Software: Are We There Yet? *Empirical Software Engineering*. 2022, vol. 28, no. 1, p. 13. Available from doi: 10.1007/s10664-022-10247-x.

4. Blackburn, Stephen M.; Garner, Robin; Hoffmann, Chris; Khang, Asjad M.; McKinley, Kathryn S.; Bentzur, Rotem; Diwan, Amer; Feinberg, Daniel; Frampton, Daniel; Guyer, Samuel Z.; Hirzel, Martin; Hosking, Antony; Jump, Maria; Lee, Han; Moss, J. Eliot B.; Phansalkar, Aashish; Stefanović, Darko; VanDrunen, Thomas; Dincklage, Daniel von; Wiedermann, Ben. The DaCapo Benchmarks: Java Benchmarking Development and Analysis. In: *Proceedings of OOPSLA 2006*. 2006. Available from doi: 10.1145/1167473.1167488.

5. Sewe, Andreas; Mezini, Mira; Sarimbekov, Aibek; Binder, Walter. Da Capo Con Scala: Design and Analysis of a Scala Benchmark Suite for the Java Virtual Machine. In: *Proceedings of OOPSLA 2011*. 2011. Available from doi: 10.1145/2048066.2048118.

6. Prokopec, Aleksandar; Rosà, Andrea; Leopoldseder, David; Duboscq, Gilles; Tůma, Petr; Studener, Martin; Bulej, Lubomír; Zheng, Yudi; Villazón, Alex; Simon, Doug; Würthinger, Thomas; Binder, Walter. Renaissance: Benchmarking Suite for Parallel Applications on the JVM. In: *Proceedings of PLDI 2019*. 2019. Available from doi: 10.1145/3314221.3314637.

7. Yue, Zhihan; Wang, Yujing; Duan, Juanyong; Yang, Tianmeng; Huang, Congrui; Tong, Yunhai; Xu, Bixiong. TS2Vec: Towards Universal Representation of Time Series. In: *Proceedings of the AAAI Conference on Artificial Intelligence*. 2022, vol. 36, pp. 8980–8987. No. 8.

8. Oord, Aäron van den; Li, Yazhe; Vinyals, Oriol. Representation Learning with Contrastive Predictive Coding. *arXiv preprint arXiv:1807.03748*. 2018.

9. Franceschi, Jean-Yves; Dieuleveut, Aymeric; Jaggi, Martin. Unsupervised Scalable Representation Learning for Multivariate Time Series. In: *Advances in Neural Information Processing Systems*. 2019, vol. 32.

10. Tonekaboni, Sana; Eytan, Danny; Goldenberg, Anna. Unsupervised Representation Learning for Time Series with Temporal Neighborhood Coding. In: *International Conference on Learning Representations*. 2021.

List of Figures

List of Tables

List of Abbreviations

A Attachments

A.1 First Attachment